

SVILUPPO DI UN COVERT CHANNEL ICMP PER L'ESFILTRAZIONE DEI DATI

F. Santini, M. Mecarelli

Perugia – February 1, 2026

Available at <https://github.com/mcrrmc/Tesi-Magistrale---M.-Mecarelli-F.-Santini.git>

INTRODUZIONE

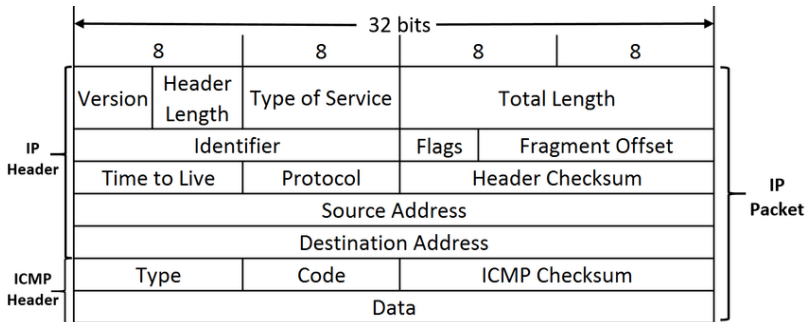
La sicurezza informatica rappresenta oggi una delle sfide più critiche nell'ambito delle comunicazioni e delle reti informatiche. Data la continua evoluzione delle tecnologie e dato il costante aumento nella complessità dei sistemi informatici; emergono costantemente nuove vulnerabilità e tecniche di attacco che permettono l'esfiltrazione dei dati.

Tra queste, i Covert Channel costituiscono una minaccia particolarmente insidiosa, in quanto permettono la trasmissione di informazioni (fra due entità) ma la cui individuazione risulta complessa da parte dei tradizionali sistemi di sicurezza.

INTERNET CONTROL MESSAGE PROTOCOL

ICMP è un protocollo che opera al livello di rete (livello 3 modello ISO/OSI), e permette la segnalazione di errori, la diagnostica della rete e la messaggistica di controllo. Date queste caratteristiche risulta fondamentale per il funzionamento, il monitoraggio e la diagnostica delle reti IP e che, proprio per questo, non può essere completamente disabilitato o filtrato nelle reti moderne senza comprometterne l'operatività. Tutto ciò lo rende un candidato ideale per la creazione di canali di comunicazione nascosti e difficilmente rilevabili.

Successivamente verrà mostrato come il protocollo è stato sfruttato per l'implementazione di diverse tipologie di Covert Channel.



Struttura di un messaggio ICMP

Utilizzi	Descrizione
Diagnostica della rete	Fornisce metriche critiche per il monitoraggio continuo delle prestazioni della rete
Segnalazione di errori	Rileva e segnala i problemi riscontrati durante la trasmissione dei dati tra i dispositivi sulla rete
Risoluzione dei problemi	Gli amministratori di rete possono ricevere avvisi in tempo reale e rispondere rapidamente ai problemi di rete grazie agli strumenti di monitoraggio basati su ICMP.
Ottenimento di informazioni	Invia messaggi e informazioni senza la necessità di una connessione preventiva.

Utilizzi del protocollo ICMP

COVERT CHANNEL

Un **Covert Channel** è un attacco che permette (in ambienti ritenuti sicuri) la capacità di comunicare e/o trasferire dati in maniera non autorizzata e non voluta. Solitamente operano al di fuori degli usuali meccanismi di comunicazioni sfruttando vulnerabilità o comportamenti non previsti nei sistemi. Ciò gli permette di non generare segnali di un uso improprio del sistema ed inoltre, nascondendosi all'interno dei normali processi del sistema, sono difficili da rilevare e/o identificare.

Qualsiasi risorsa condivisa può essere utilizzata per la creazione di un canale nascosto.

CARATTERISTICHE DI UN COVERT CHANNEL

Per poter essere efficace, un Covert Channel deve possedere determinate caratteristiche. Fra queste, l'**indistinguibilità** è quella principale.

Caratteristica	Descrizione
Furtività	Evitare di attirare le attenzioni sia degli amministratori che degli strumenti di rilevamento degli attacchi.
Capacità di trasmissione	La quantità di dati che il canale trasmette in un determinato intervallo di tempo. Una quantità elevata può portare ad una rilevazione del canale.
Uso delle risorse	Un uso delle risorse improprio (o sproporzionato) potrebbe andare in conflitto con le risorse legittime presenti nel sistema.
Rumore	L'uso dei servizi e/o delle risorse nel sistema potrebbe alterare il loro funzionamento.
Indistinguibilità	La trasmissione dei dati mantiene conforme e inalterato il funzionamento delle risorse utilizzate. Si riesce a rendersi indistinguibili dalla risorsa autorizzata.

Caratterisitche di un Covert Channel

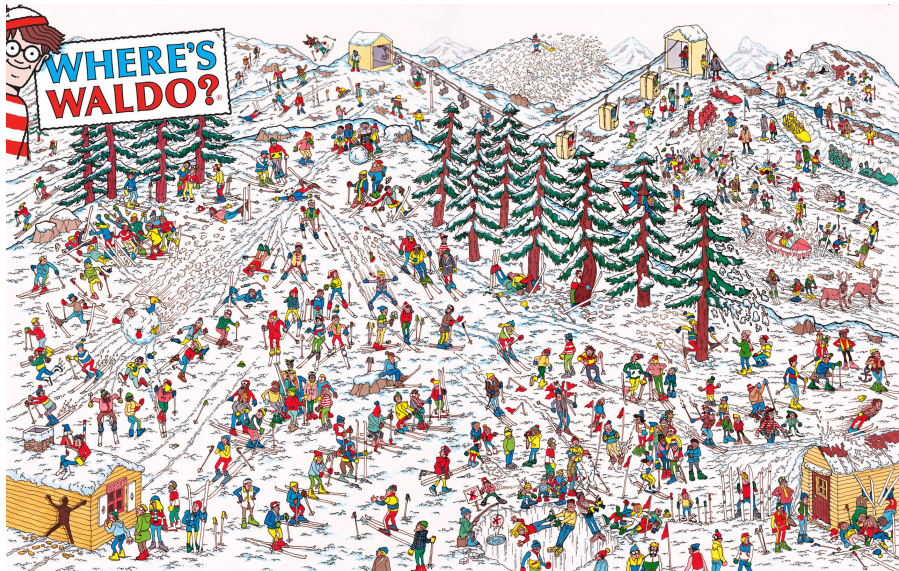
FURTIVITÀ



CAPACITÀ DI TRASMISSIONE



INDISTINGUIBILITÀ



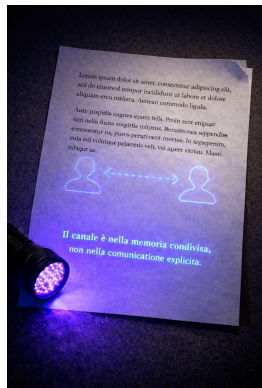
TIMING COVERT CHANNEL

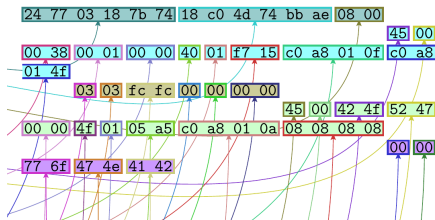
I **Timing Covert Channel** sono metodi di comunicazione che permettono ad un osservatore di acquisire informazioni attraverso il cambiamento del tempo di risposta di una risorsa. Qualsiasi metodo che utilizza un orologio (o una misurazione del tempo) per segnalare il valore può implementarlo.



STORAGE COVERT CHANNEL

Uno **Storage Covert Channel** viene creato scrivendo dei dati su un'area di memoria condivisa accessibile da tutte entità presenti. I veicoli saranno tutte quelle risorse che consentiranno la scrittura (diretta o indiretta) da parte di un processo e la lettura (diretta o indiretta) da parte di un altro.





Hex dump di un pacchetto

```
$w..{t..Mt....E.
.8....@.....
.0.....E.BORG
..0.....
woGNAB
```

Decodifica ASCII di un pacchetto

BEHAVIOURAL COVERT CHANNEL

Nei **Behavioural Covert Channel** le informazioni vengono codificate modificando il comportamento del sistema. Quindi i dati vengono ricavati osservando il comportamento del sistema piuttosto che attraverso l'accesso ai dati.



Alcuni strumenti analizzati sono:

- Virtualbox
- Scapy
- RITA

Invece alcuni progetti analizzati sono:

- ICMP door
- ICMP exfil
- ICMP tunnel

STRUMENTI UTILIZZATI

VirtualBox è un software di virtualizzazione open source che consente di eseguire più sistemi operativi (guest) su un singolo computer (host). È stato usato per la creazione di macchine virtuali Ubuntu; così da testare il codice sviluppato in quell'ambiente.

Scapy è un framework per la manipolazione dei pacchetti scritto in Python. Consente di falsificare varie tipologie di pacchetti Inoltre può inviarli in rete, catturarli, memorizzarli o leggerne i dati. La presenza di metodi che permettono l'inizio e la fine dello sniffing; ha permesso lo sviluppo dei canali di comunicazione tramite protocollo ICMP.

RITA è uno strumento open source per la ricerca delle minacce di rete, progettato per identificare attività di comando e controllo (C2) dannose. Acquisisce i log di Zeek e utilizza l'analisi comportamentale per identificare sistemi potenzialmente compromessi.

ICMP DOOR

È stato studiato per comprendere come poter strutturare il tunneling dei dati. Oltre alla struttura delle entità, che successivamente verranno ridefinite, risulta interessante che il programma richieda degli argomenti dall'utente. Tramite la libreria *argparse* richiede l'interfaccia su cui ascoltare i dati e l'indirizzo di destinazione dei pacchetti. Inoltre tramite il metodo *sniff* del framework Scapy ascolta il flusso dei dati mentre con *sr* invia i pacchetti.

Alcuni svantaggi saranno che i dati vengono trasmessi in chiaro nel campo Data del messaggio ICMP Echo Reply. Inoltre il campo identifier rimarrà invariato per tutta la comunicazione e siccome a molteplici richieste non potrà essere associata una richiesta. Un sistema di sicurezza potrebbe bloccare i pacchetti.

ICMP EXFIL

Si è analizzato ICMP Exfil per vedere come un Covert Channel temporalizzato potesse funzionare. Riceve un dato, lo converte in binario e dopodichè avrà una lista di numeri binari. Per mandare il dato, invia un ping con un timeout pari a **binary_number+leway**. Il valore leway viene utilizzato per rallentare il numero di pacchetti inviati ed avere una connessione maggiormente silenziosa. L'autore infine indica come miglioria, la crittografia dei dati per aggiungere del rumore, dell'entropia. Ciò su cui si affida è che un osservatore, vedendo i pacchetti ICMP, li veda come validi; Siccome non riuscirebbero a trovare alcun dato, a meno che non sappiano della tecnica utilizzata.

ICMP TUNNEL

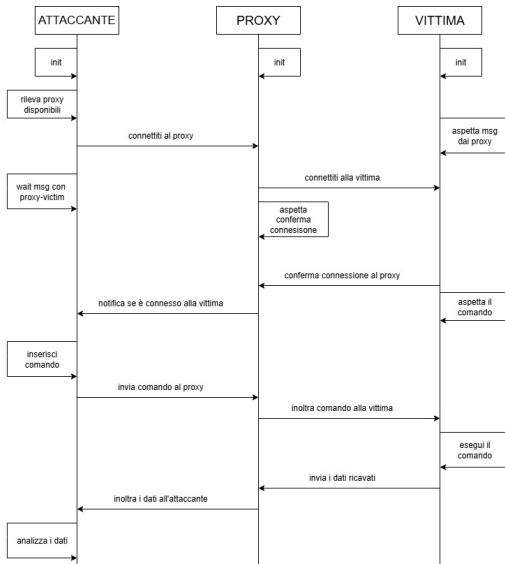
È uno strumento che permette il tunneling del traffico IP. Tramite delle richieste e risposte ICMP Echo, incapsula il traffico e lo invia al server proxy. Quest'ultimo lo decapsula e lo inoltra. I pacchetti in entrata, che sarebbero diretti alla macchina vittima, sono poi incapsulati dal proxy e inviati. L'approccio è possibile siccome RFC-792, che indica le linee guida del protocollo ICMP, permette una quantità arbitraria di dati nei pacchetti ICMP Echo (sia richiesta che risposta).

IMPLEMENTAZIONE

Alcuni aspetti relativi all'implementazione della comunicazione, sono stati ricavati analizzando i progetti che già hanno affrontato il problema:

1. La **presenza di più proxy intermediari** fra l'attaccante e la vittima. Così da distribuire il carico sulla rete dato dall'invio dei dati.
2. Un limite alla **quantità di dati** inviabili in un intervallo di tempo.
3. Trasmettere i **dati in chiaro** permette a chiunque di leggerne il contenuto. Mentre la trasmissione di dati cifrati potrebbe destare sospetti.
4. I **messaggi di tipo Request**, inviano un messaggio di richiesta e poi aspettano una risposta; che sarà uguale al messaggio inviato. Una soluzione è l'invio solo di messaggi di tipo Reply oppure la disabilitazione delle risposte (e inviare noi stessi una risposta).

STRUTTURA DELA COMUNICAZIONE



Flusso di comunicazione fra le entita

STRUTTURA DELL'ATTACCANTE

Tramite un parser rileva se nella linea di comando sono state inserite le opzioni necessarie alla sua inizializzazione:

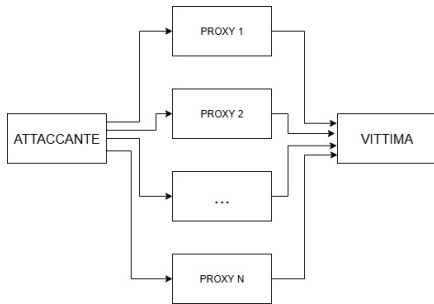
Opzione	Utilizzo
Path file	Percorso per il file di configuraizone da caricare. In questo file JSON viene specificato l'indirizzo IP della vittima, la metodologia di attacco e la lista dei proxy che si vogliono usare.

Da questo carica un file di configurazione nel quale viene definito l'**indirizzo IP della vittima**, il **metodo di attacco** e i **proxy utilizzabili** per farlo.

Procede poi a verificare quali proxy (fra quelli presenti) siano disponibili; ovvero quali riescono a comunicare sia con l'attaccante che con la vittima. Ciò viene effettuato stabilendo una connessione o tramite protocollo TCP/IP o ICMP (lo stesso che il proxy userà con la vittima) e invia un messaggio che indica l'intento di stabilire un canale di comunicazione. Rimane poi in attesa che il proxy confermi la connessione.

Dopodiché inserirà il comando che la vittima dovrà eseguire (o il dato che gli si vuole mandare). Nel caso utilizzasse dei proxy, aspetterà che essi inoltrino i dati che la vittima ha inviato a ciascuno di essi. Una volta ricevuti tutti i dati dai proxy, l'attaccante li riordinerà per ricavare il messaggio correttamente. Infine l'attaccante potrà decidere se inviare un ulteriore comando o se terminare la comunicazione. In quest'ultimo caso, notificherà l'evento a ciascuno dei proxy.

Per la comunicazione l'attaccante potrà usare uno (nel caso standard) o più proxy per comunicare con la vittima. In questo caso si prevederà un metodo per poter riordinare i messaggi che l'attaccante ha ricevuto dai proxy.



Tuttavia sarà possibile (per l'attaccante) anche comunicare direttamente con la vittima. In questo caso alcune funzioni presenti nel proxy verranno eseguite dall'attaccante (e.g connettersi alla vittima).

STRUTTURA DEL PROXY

Durante la fase di inizializzazione, il proxy ha bisogno di sapere l'indirizzo IP dell'attaccante. Questo così da poter accettare solamente la sua connessione; tutte le altre verranno ignorate o scartate.

Opzione	Utilizzo
Indirizzo IP	Rappresenta l'indirizzo IP dell'attaccante.

Una volta stabilita la connessione con l'attaccante; che può avvenire sia tramite il protocollo TCP/IP che ICMP, aspetterà che gli venga indicato l'indirizzo IP della vittima, oltre al metodo di attacco scelto.

Ricevuto l'indirizzo ci si assicura che sia possibile comunicare con la vittima. In caso negativo l'attaccante verrà aggiornato che il proxy non sarà utilizzabile per l'filtrazione dei dati.

Rimane poi in ascolto che l'attaccante gli invii il comando per la vittima oppure gli notifichi di aspettare direttamente i dati. Questo perchè solo un proxy riceverà il comando da inoltrare. Aspetta poi che la vittima inizi ad inviare i dati da esfiltrare; i quali verranno inoltrati all'attaccante.

Un volta terminato, rimarrà in ascolto di aggiornamenti da parte dell'attaccante; che potranno essere o un nuovo comando oppure la notifica sulla chiusura della comunicazione. Nel secondo caso provvederà ad aggiornare la vittima della cosa.

STRUTTURA VITTIMA

La vittima dovrà sapere di quanti proxy connessi ha bisogno (il cui valore minimo è uno). Essi sono necessari sia la ricezione dell comando sia per l'esfiltrazione dei dati.

Opzione	Utilizzo
Numero dei Proxy	Numero minimo di proxy necessari per l'esecuzione dell'attacco.

La vittima rimane in attesa di connessioni da parte dei proxy (o dell'attaccante). Una volta definiti tutti i proxy utilizzabili per la comunicazione, aspetterà di ricevere da uno di loro un comando.

Una volta ricevuto lo eseguirà ed i dati ricavati (sia quelli relativi all'output che ad eventuali errori) verranno inviati all'attaccante tramite i proxy connessi.

IMPLEMENTAZIONE TIMING COVERT CHANNEL

Per poter temporizzare i dati da inviare, si sono definiti vari metodi per la codifica e decodifica degli intervalli temporali. Dovrà quindi essere presente una funzione che è responsabile di mappare un dato binario a un intervallo di tempo.

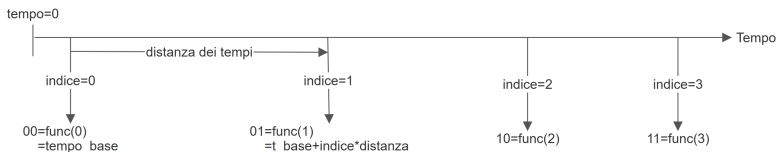
Per la **codifica**, si calcolerà il delay temporale associato al dato da mandare; esso verrà usato come il tempo da aspettare prima di inviare un pacchetto.

Per la **decodifica**, il destinatario rimane in attesa dei pacchetti e ogni volta calcolerà la differenza di tempo fra il pacchetto precedente e quello attuale. Dall'intervallo temporale fra i due pacchetti, ricaverà il dato relativo.

PRIMA IMPLEMENTAZIONE DELLA FUNZIONE

In una **prima versione**, si associa ad ogni punto una distanza di sicurezza, coì da evitare che due codici combacino verso lo stesso tempo di delay.

$$\text{tempo_base} + \text{indice} * \text{distanza_tempi}$$

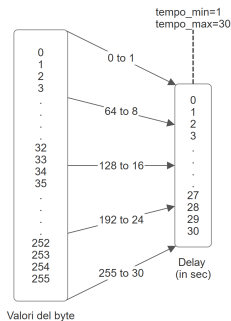


Le distanze fra i vari intervalli di tempo sono necessarie per evitare errori nella decodifica siccome durante la trasmissione del pacchetto possono avvenire dei ritardi sui datagram inviati.

SECONDA IMPLEMENTAZIONE DELLA FUNZIONE

Lo svantaggio dell'approccio precedente, sono i tempi di attesa che sono dati dal fatto che il tempo di delay è basato sul valore che lo stesso byte ha.

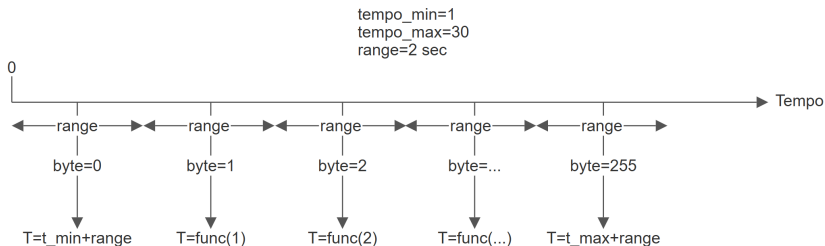
Quindi si è definita una funzione che normalizzerà l'intervallo di tempo (associato al dato), fra un valore minimo ed uno massimo. Tramite questo approccio, il range dei delay viene ristretto ad un intervallo preciso e definito.



$$\text{min_delay} + (\text{byte}/255) * (\text{max_delay} - \text{min_delay})$$

RANDOMIZZAZIONE DEI TEMPI DI INVIO

Si evolve il Covert Channel precedentemente aggiungendo del rumore per permetterla di far sembrare casuale il tempo di delay.



Questo porta alla necessita che, per potersi scambiare i dati, il mittente e il destinatario devono sincronizzarsi sulla quantità di rumore aggiunto.

IMPLEMENTAZIONE STORAGE COVERT CHANNEL

Siccome la risorsa condivisa sarà il pacchetto ICMP, il destinatario, potrà ricavare i dati leggendo i valori presenti nei suoi campi.

Tipologia	Byte per pacchetto	Campi Sfruttatti
Packet Too Big	40+8+41 →min 89 byte	mtu invoking packet
Destination Unreachable Time Exceeded	40+8+41 →min 89 byte	unused invoking packet
Parameter Problem	40+8+41 →min 89 byte	pointer invoking packet
Echo Request/Reply	40+8+32 →min 80 byte	identifier data

Tipologie di messaggi ICMPv6 usati

Tipologia	Byte per pacchetto	Campi Utilizzati
Redirect Message	20+8+21 →min 49 byte	header+64 bits
Destination Unreachable Time Exceeded Source Quench	20+8+21 →min 49 byte	unused header+64 bits
Parameter Problem	20+8+21 →min 49 byte	pointer unused header+64 bits
Information Request/Reply	20+8 →28 byte	identifier
Echo Request/Reply	20+8+32 →min 60 byte	identifier data
Timestamp Request/Reply	20+8+12+32 →min 72 byte	identifier timestamp data

Tipologie di messaggi ICMPv4 usati

REDIRECT

Il campo utilizzato per l'inserimento dei dati sarà **Internet Header+ 64 bits**.

La quantità di dati esfiltrabili tramite questo messaggio sarà pari a **9 byte**.

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

Type=5 (1 byte)	Code=0-3 (1 byte)	Checksum (2 byte)
Gateway Internet Address (4 byte)		
Internet Header + 64 bits of Original Datagram ($\geq 21\text{B}$)		

Struttura del pacchetto **ICMPv4 Redirect**

Ethernet

dst
src
type

IP

version
ihl
tos
len
id
flags
frag
ttl
proto
chksum
src
dst
options

ICMP

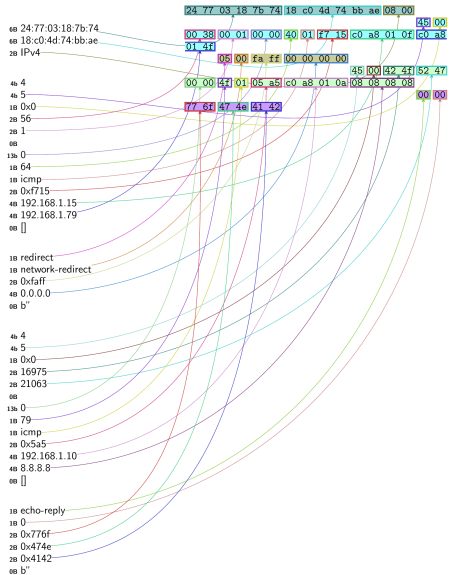
type
code
chksum
gw
unused

IP in ICMP

version
ihl
tos
len
id
flags
frag
ttl
proto
chksum
src
dst
options

ICMP in ICMP

type
code
chksum
id
seq
unused



Messaggio ICMP Redirect

INSERIMENTO DEI DATI NEL CAMPO HEADER+64 BITS

Nel campo sono presenti l'header IP più 8 byte per una lunghezza complessiva di 28 byte.

Nel nostro caso si inserirà un pacchetto ICMP/IP; sebbene si possono usare anche altre combinazioni di protocolli (e.g TCP/IP). Inoltre siccome non rappresenta un datagram da inviare ma già spedito, si possono sfruttare un maggior numero di campi.

IP in ICMP

version	4b	4
ihl	4b	5
tos	1B	0x0
len	2B	16975
id	2B	21063
flags	0B	
frag	13b	0
ttl	1B	79
proto	1B	icmp
chksum	2B	0x5a5
src	4B	192.168.1.10
dst	4B	8.8.8.8
options	0B	[]

ICMP in ICMP

type	1B	echo-reply
code	1B	0
chksum	2B	0x776f
id	2B	0x474e
seq	2B	0x4142
unused	0B	b"

Messaggio ICMP con campo
Header+64 bit impostato

Header IPv4

Campo	Byte	Utilizzo
Time to live	1 byte	Tempo di vita del pacchetto
Total length	2 byte	Dimensione dell'intero pacchetto
Identification	2 byte	Identifica i frammenti di un pacchetto IP

Header ICMPv4

Campo	Byte	Utilizzo
Identifier	2 byte	Identificativo del messaggio di richiesta invocato
Sequence	2 byte	Numero di sequenza del messaggio di richiesta invocato

Uso del Header+64 bits nel caso di IP/ICMP

DESTINATION UNREACHABLE

Sfrutta i campi **unused** e **header+64 bit**. Per una capacità di esfiltrazione pari a **9+4 bytes**.

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

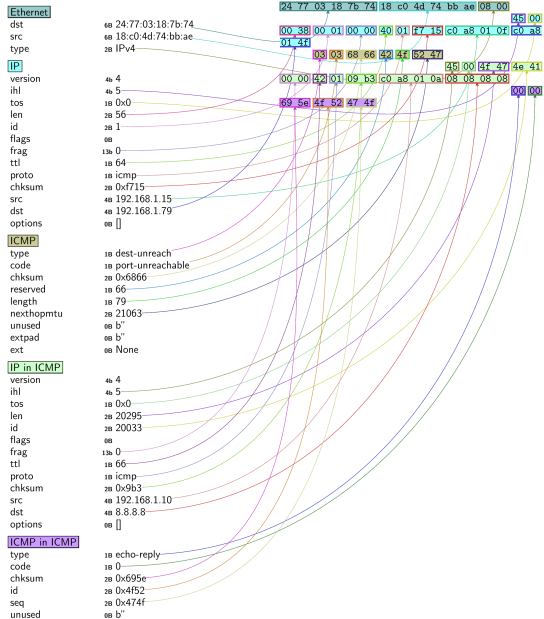
Type=3 (1 byte)	Code=0-5 (1 byte)	Checksum (2 byte)
Unused (4 byte)		
Internet Header + 64 bits of Original Datagram (≥ 21 byte)		

Struttura del pacchetto **ICMPv4 Destination Unreachable**

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

Type=1 (1 byte)	Code=0-6 (1 byte)	Checksum (2 byte)
Unused (4 byte)		
As much of invoking packet as possible without		
the ICMPv6 packet exceeding the minimum IPv6 MTU (≥ 0 byte)		

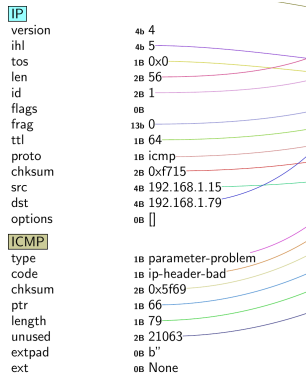
Struttura del pacchetto **ICMPv6 Destination Unreachable**



Messaggio ICMP Destination Unreachable

INSERIMENTO DEI DATI NEL CAMPO UNUSED

Dalle specifiche RFC (792 e 4434) il campo deve essere 0. Quindi Scapy, per inviarlo conforme allo standard, azzererà qualsiasi valore al suo interno. Il problema può essere risolto tramite degli accorgimenti. Tramite la libreria **struct** è possibile costruire un pacchetto avente i dati all'interno del campo e si userà Scapy solo per l'invio.



Messaggio ICMP con campo *unused* impostato

Siccome il suo utilizzo rende il messaggio non conforme agli standard RFC, un IDS che implementi la Deep Packet Inspection rileverà l'anomalia.

PARAMETER PROBLEM

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

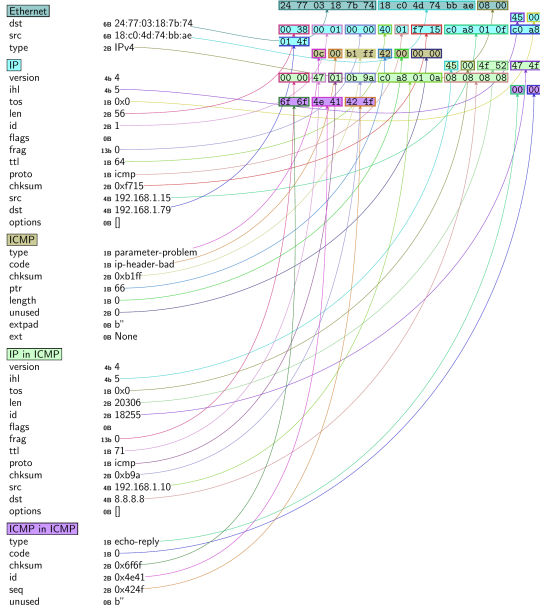
Type=12 (1 byte)	Code=0 (1 byte)	Checksum (2 byte)
Pointer (1 byte)	Unused (3 byte)	
Internet Header + 64 bits of Original Datagram (≥ 21 byte)		

Struttura del pacchetto **ICMPv4 Parameter Problem**

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

Type=4 (1 byte)	Code=0-2 (1 byte)	Checksum (2 byte)
Pointer (4 byte)		
As much of invoking packet as possible without		
the ICMPv6 packet exceeding the minimum IPv6 MTU (≥ 0 byte)		

Struttura del pacchetto **ICMPv6 Parameter Problem**



Messaggio ICMP Parameter Problem

INFORMATION REQUEST / INFORMATION REPLY

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
Type=15 (1 byte)								Code=0 (1 byte)								Checksum (2 byte)															
Identifier (2 byte)																Sequence Number (2 byte)															

Struttura del pacchetto **ICMPv4 Information**

Il campo *Identifier* definisce l'identificativo delle richieste e può assumere un qualsiasi valore. Il campo *Sequenza* (dalle specifiche RFC 792), viene incrementato ad ogni richiesta inviata con lo stesso identificativo.

ICMP

type
code
chksum
id
seq
unused

1B information-respon[...]
1B 0
2B 0xad**b0**
2B 0x424**f**
2B 0x0
0B b''

Messaggio ICMP con campo *identifier* impostato

#PACCHETTO No1

0000	24	77	03	18	7B	74	18	C0	4D	74	BB	AE	08	00	45	00	\$w..{t..Mt....E.
0010	00	1C	00	01	00	00	40	01	F7	31	C0	A8	01	0F	C0	A8	@..1.....
0020	01	4F	10	00	AD	B0	42	4F	00	00							.O....BO..

#PACCHETTO No2

0000	24	77	03	18	7B	74	18	C0	4D	74	BB	AE	08	00	45	00	\$w..{t..Mt....E.
0010	00	1C	00	01	00	00	40	01	F7	31	C0	A8	01	0F	C0	A8	@..1.....
0020	01	4F	10	00	9D	B8	52	47	00	00							.O....RG..

#PACCHETTO No3

0000	24	77	03	18	7B	74	18	C0	4D	74	BB	AE	08	00	45	00	\$w..{t..Mt....E.
0010	00	1C	00	01	00	00	40	01	F7	31	C0	A8	01	0F	C0	A8	@..1.....
0020	01	4F	10	00	A0	B8	4F	47	00	00							.O....OG..

#PACCHETTO No4

0000	24	77	03	18	7B	74	18	C0	4D	74	BB	AE	08	00	45	00	\$w..{t..Mt....E.
0010	00	1C	00	01	00	00	40	01	F7	31	C0	A8	01	0F	C0	A8	@..1.....
0020	01	4F	10	00	A1	BE	4E	41	00	00							.O....NA..

Dump esadecimale di quattro messaggi Information Request

Ethernet

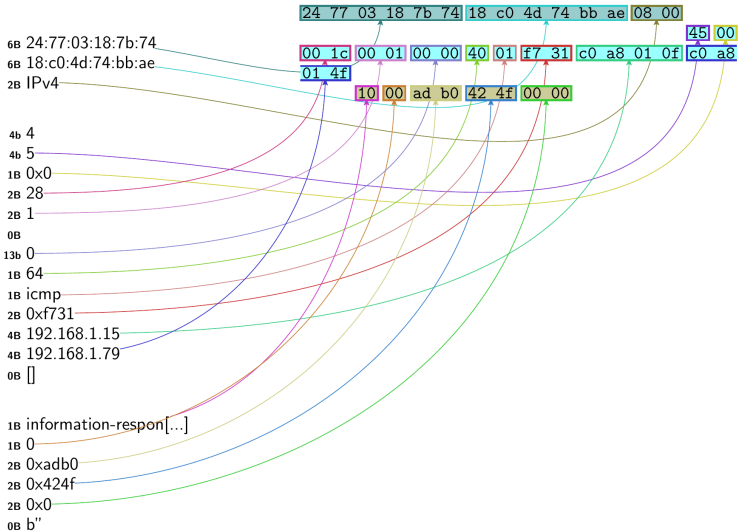
dst
src
type

IP

version
ihl
tos
len
id
flags
frag
ttl
proto
chksum
src
dst
options

ICMP

type
code
chksum
id
seq
unused



Messaggio ICMP Information Request/Reply

ECHO REQUEST/REPLY

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
Type=8 (1 byte)								Code=0 (1 byte)								Checksum (2 byte)															
Identifier (2 byte)																Sequence Number (2 byte)															
Data ... (≥ 0 byte)																															

Struttura del pacchetto **ICMPv4 Echo**

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
Type=128 (1 byte)								Code=0 (1 byte)								Checksum (2 byte)															
Identifier (2 byte)																Sequence Number (2 byte)															
Data ... (≥ 0B)																															

Struttura del pacchetto **ICMPv6 Echo**

INSERIMENTO DEI DATI NEL CAMPO DATA

Nel campo il mittente può inserire quanti dati preferisce. Ma per sicurezza la dimensione sarà o 32 o 64 byte. Inoltre una quantità di dati eccessiva può far mandare più pacchetti di risposta nel caso si fosse inviata una richiesta.

INSERIRE IMMAGINE IN CUOI SI
MANDA NEL CMAPO DATA UN SACCO
DI DATI TALE CHE SI SAVRANNO
TANTE REPLY

ICMP

type
code
chksum
id
seq
unused

Raw

load

1B echo-reply
1B 0
2B 0x6f05
2B 0x1
2B 0x0
0B b''

64B b'BORGOGNABORGOGNA[...]

Messaggio ICMP con campo *Data*
impostato

TIMESTAMP REQUEST/REPLY

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
Type=13 (1 byte)								Code=0 (1 byte)								Checksum (2 byte)															
Identifier (2 byte)																Sequence Number (2 byte)															
Originate Timestamp (4 byte)																															
Receive Timestamp (4 byte)																															
Transmit Timestamp (4 byte)																															
Data ... (≥ 0 byte)																															

Struttura del pacchetto **ICMPv4 Timestamp**

Contiene i millisecondi dalla mezzanotte UT e ciascun campo ha un ordine temporale specifico. Che dovrà rimanere congruente anche dopo che si sono inseriti i dati.

Dato nascosto: b'r' 114
Dato nascosto: b'g' 103
Dato nascosto: b'o' 111

Timestamp origin prima:
2025-11-07 23:13:36.146254+00:00

Timestamp origin dopo:
2025-11-07 23:13:36.114000+00:00

Tempo nel campo:
83616114

Esempio di un dato inserito nel
campo Timestamp

ICMP

type	1B timestamp-reply
code	1B 0
chksum	2B 0xb3
id	2B 0x424f
seq	2B 0x0
ts_ori	4B 22:42:16.82
ts_rx	4B 22:42:17.71
ts_tx	4B 22:42:18.79
unused	0B b"

Messaggio ICMP con i campi
Timestamp impostati

Ethernet

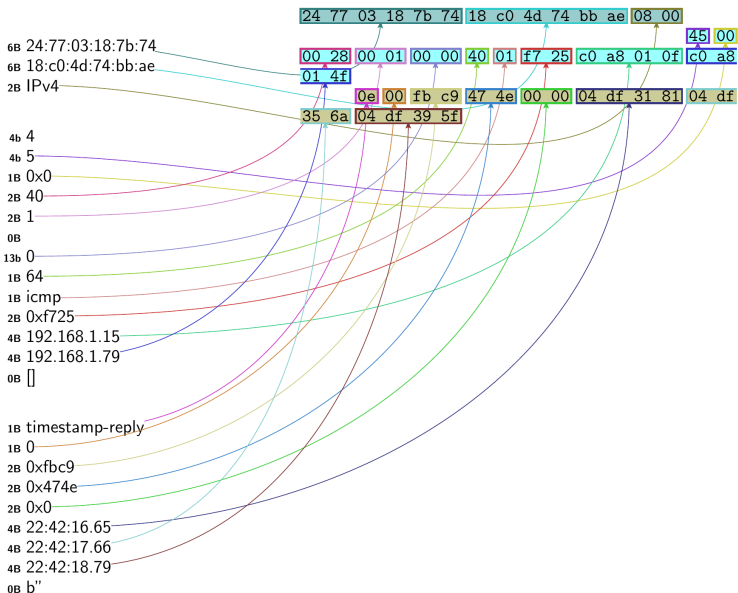
dst
src
type

IP

version
ihl
tos
len
id
flags
frag
ttl
proto
chksum
src
dst
options

ICMP

type
code
chksum
id
seq
ts_ori
ts_rx
ts_tx
unused



Messaggio ICMP Timestamp Request/Reply

INSERIMENTO DEI DATI NEL CAMPO TIMESTAMP

In ciascun campo timestamp verrà inserito un byte nella parte dei milisecondi. Non è possibile inserire ulteriori informazioni nelle altre parti del campo siccome rappresentano le ore, i minuti ed i secondi. Un'alternativa è l'inserire il dato in tutto il campo ma, rappresentando un valore temporale, ciò porterebbe al rischio di avere tempi errati o non congruenti. Per esempio nel pacchetto viene risultare ricevuto prima che sia stato inviato.

ICMP

type
code
chksum
id
seq
ts_ori
ts_rx
ts_tx
unused

1B timestamp-reply
1B 0
2B 0xb3
2B 0x424f
2B 0x0
4B 22:42:16.82
4B 22:42:17.71
4B 22:42:18.79
0B b"

Messaggio ICMP Timestamp
Request/Reply

PACKET TOO BIG

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
Type=2 (1 byte)								Code=0 (1 byte)								Checksum (2 byte)															
MTU (4 byte)																															
As much of invoking packet as possible without																															
the ICMPv6 packet exceeding the minimum IPv6 MTU (≥ 0 byte)																															

Struttura del pacchetto **ICMPv6 Packet Too Big**

Nel campo **MTU** viene indicata la capacità massima del collegamento.
L'intera dimensione del campo verrà usata per l'inserimento dei dati.
Questo permetterà di poter inserire **4 byte** di dati.

Invece nel campo **Invoking Packet** verrà indicato il pacchetto che ha causato l'errore. Nel nostro caso si userà un pacchetto IP/ICMP. Tramite i campi sfruttati si riesce ad inserire **7 byte** di dati.

Header IPv6

Campo	Byte	Utilizzo
Payload Length	2 byte	Tempo di vita del pacchetto
Hop Limit	1 byte	Decrementato di 1 per ogni nodo che inoltra il pacchetto

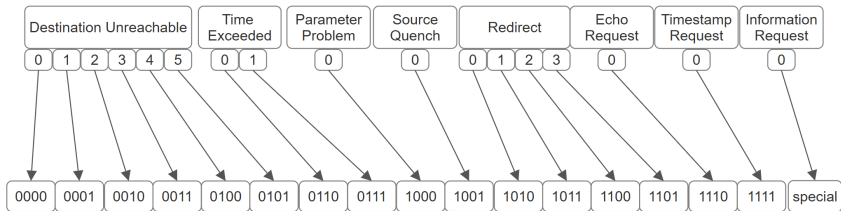
Header ICMPv6

Campo	Byte	Utilizzo
Identifier	2 byte	Identificativo del messaggio di richiesta invocato
Sequence	2 byte	Numero di sequenza del messaggio di richiesta invocato

Struttura del campo Invoking Packet

IMPLEMENTAZIONE BEHAVIOURAL COVERT CHANNEL

Ciò che indicherà l'informazione trasmessa, è l'arrivo di un determinato messaggio ICMP (la cui tipologia e codice indicano il dato). Siccome il numero di eventi possibili sono 17, ognuno permetterà di codificare 4 bit alla volta ($\log_2 16 = 4$). Quindi l'invio di un byte, prevede la sua divisione in due parti e l'avvenimento di due eventi.



Per ricavare i dati, si monitora il traffico di rete e si controllano le tipologie di messaggi ICMP ricevuti. Da ognuno si ricaveranno i quattro bit.

```
data='m'  
data.hex=0x6D  
data.bytes=b'0110 1101'
```

#PRIMO EVENTO

Maschera il byte con b'11110000' per ottenere: 0x60
Effettua lo shift a destra per ottenere: 0x06 (0110)
Si ottiene come tipologia: Time Exceeded (tipo 11)
Si ottiene come codice: 0 (Time to Live exceeded in Transit)
Invia un messaggio ICMP Time Exceeded con codice 0

#SECONDO EVENTO

Maschera il byte con b'00001111' per ottenere: 0x0D (1101)
Si ottiene come tipologia: Redirect (tipo 5)
Si ottiene come codice: 3 (Redirect for the host)
Invia un messaggio ICMP Redirect con codice 3

Esempio dell'invio di un byte

TIPOLOGIE DI MESSAGGI DEPRECATI

L'uso di tipologie deprecate non assicura la ricezione dei messaggi, siccome i router o firewall potrebbero ignorarli. Inoltre se inviati ripetutamente potrebbero destare dei sospetti sul flusso di comunicazione.

Le tipologie deprecate sono:

- Information Request/Reply: che è stato sostituito da protocolli come il DHCP.
- Source Quench: che era concepito come meccanismo per il controllo della congestione ma, data la sua inefficienza, è stato sostituito dai meccanismi presenti nel protocollo TCP.

TEST E RISULTATI

I test potranno variare in base a determinate caratteristiche:

- La **quantità di dati inviati** (che saranno 1KB, 10KB, 100KB, 1MB).
- I **campi usati** dal Covert Channel nei messaggi ICMP.
- Il **delay fra i blocchi di dati**. Si può decidere se avere un delay fra l'invio di un blocco e il successivo oppure un invio sequenziale dei dati.
- Tempo di **attesa prima di un nuovo invio** dei dati (che potrà essere 1 ora, 30 minuti, 15 minuti).
- Il **mittente del messaggio**. Si può usare il mittente reale oppure si inserisce come mittente l'indirizzo IP di un altro host.

Il Covert Channel utilizzato durante la fase di test è quello che utilizza i messaggi ICMP ECHO.

TEST CON SINGOLO INVIO DEI DATI

In questi test i dati sono stati mandati una singola volta. I blocchi dei dati sono da 1024 bytes ($\approx$ 1KB).

	1 KB	10 KB	100 KB	1 MB
Campo ID	Non presente	Non presente	Non presente	Non presente
Campo ID + delay	Non presente	Non presente	Non presente	Low
Campo ID + payload fisso	Non presente	Non presente	Non presente	Non presente
Campo ID + payload fisso + delay	Non presente	Non presente	Non presente	Low
Payload fisso	Non presente	Non presente	Non presente	Non presente
Payload fisso + delay	Non presente	Non presente	Non presente	Low
Payload randomico	Non presente	Non presente	Non presente	None
Payload randomico + delay	Non presente	Non presente	Non presente	Low

Test tramite un singolo invio dei dati

TEST CON INVII MULTIPLI DEI DATI

In questi test si invierà per tre volte lo stesso dato. L'esfiltrazione dei dati avverrà tramite messaggi ICMP Echo sfruttando il campo payload.

Quest'ultimo, rispetto al campo Identifier, riesce a contenere una quantità maggiore di dati.

Dal primo test si ricavano i seguenti risultati:

- tempi di attesa estremamente brevi o estremamente lunghi sono da preferire.
- il cambio di mittenti per ogni pacchetto senza un tempo di attesa adeguato, viene visto da RITA come un impulso di comunicazione.

Covert Channel	Connessione più lunga	Etichetta	Beacon Score
Pausa di un ora e mittente reale	10h 3m 6s	Medium	14.7%
Pausa di 30 minuti e mittente reale	10h 4m 51s	Medium	37.9%
Pausa di 15 minuti e mittente reale	10h 9m 20s	Medium	0%

Pausa di un ora e mittenti falsificati	6h 52m 26s	Low	0.0%
Pausa di 30 minuti e mittenti falsificati	6h 50m 55s	Medium	13.5%
Pausa di 15 minuti e mittenti falsificati	4h 47m 22s	Low	15.8%

Test con payload fisso, 1MB di dati e un delay durante l'invio

Nel secondo test non si avrà un delay fra un blocco di dati e il successivo. I risultati sono che RITA non rileva in ogni caso la presenza di un Covert Channel. Ciò sembra più un problema relativo a come Zeek ha estratto i log cosa che porterà RITA ad un mancato rilevamento della comunicazione.

Covert Channel	Connessione più lunga	Etichetta	Beacon Score
Pausa di un ora e mittente reale	NP	NP	NP
Pausa di 30 minuti e mittente reale	NP	NP	NP
Pausa di 15 minuti e mittente reale	NP	NP	NP
Pausa di un ora e mittenti falsificati ¹	0h 1m 5s	None	33%
Pausa di 30 minuti e mittenti falsificati	NP	NP	NP
Pausa di 15 minuti e mittenti falsificati	NP	NP	NP

Test con payload fisso, 1MB di dati e nessun delay durante l'invio

Nel terzo test si inviano i100KB di dati e fra un delay fra un blocchi di dati. Come precedentemente sia il caso con il mittente reale che quello con quelli falsificati non sono stati rilevati da RITA.

Covert Channel	Connessione più lunga	Etichetta	Beacon Score
Pausa di un ora e mittente reale	NP	NP	NP
Pausa di 30 minuti e mittente reale	NP	NP	NP
Pausa di 15 minuti e mittente reale ⁰	0h 0m 3s	High	100%

Pausa di un ora e mittenti falsificati	0h 39m 42s	None	30.4%
Pausa di 30 minuti e mittenti falsificati	NP	NP	NP
Pausa di 15 minuti e mittenti falsificati	NP	NP	NP

Test con payload fisso, 100KB di dati e un delay durante l'invio

Covert Channel	Connessione più lunga	Etichetta	Beacon Score
Pausa di un ora e mittente reale	NP	NP	NP
Pausa di 30 minuti e mittente reale	NP	NP	NP
Pausa di 15 minuti e mittente reale	NP	NP	NP

Pausa di un ora e mittenti falsificati	NP	NP	NP
Pausa di 30 minuti e mittenti falsificati	NP	NP	NP
Pausa di 15 minuti e mittenti falsificati	NP	NP	NP

Test con payload randomico, 100KB di dati e un delay durante l'invio

Nel quarto test si invia 1MB di dati con un delay fra i blocchi. Inoltre i messaggi avranno un payload variabile

Covert Channel	Connessione più lunga	Etichetta	Beacon Score
Pausa di un ora e mittente reale	10h 4m 6s	Medium	0%
Pausa di 30 minuti e mittente reale	6h 49m 12s	Low	25%

I mittenti falsificati provengono da un pool di indirizzi IP attivi nella rete locale

Pausa di un ora e mittenti falsificati	6h 52m 13s	Low	32.8%
--	------------	-----	-------

I mittenti falsificati provengono da un pool di indirizzi IP non assegnati nella rete locale

Pausa di un ora e mittenti falsificati	1h 3m 15s	Low	79%
--	-----------	-----	-----

Test con payload randomico, 1MB di dati e un delay durante l'invio

RISULTATI DEI TEST

Dai risultati si nota che l'utilizzo di un singolo mittente aumenta la probabilità di essere rilevati mentre tramite la falsificazione dei mittenti il livello di gravità riuscirà minore. Tuttavia ciò ha dei difetti siccome lo scegliere per ogni pacchetto sempre un mittente diverso comporta che RITA rileverà quello scambio di informazioni come una comunicazione. Al contrario un implementazione in cui per ogni blocco si utilizza sempre lo stesso mittente (reale o falsificato) permette di diminuire la probabilità che il canale venga rilevato.

Invece il non utilizzo del delay fra l'invio dei blocchi di dati, permetta di non essere rilevati sebbene ciò può essere legato solo ad una questione di come Zeek logga i pacchetti e di come RITA elabora questi log. Tuttavia questo comporta un maggior utilizzo della banda di rete cosa che creerà del rumore da parte del Covert Channel siccome modifica il normale flusso del traffico di rete.

CONCLUSIONI FINALI SULL'IMPLEMENTAZIONE DEI COVERT CHANNEL

Questo lavoro di tesi ha affrontato lo sviluppo e l'implementazione di un covert channel basato sul protocollo ICMP per l'estrazione di dati.

L'obiettivo principale era dimostrare come il protocollo ICMP, fondamentale per il funzionamento delle reti ma spesso sottovalutato dal punto di vista della sicurezza, possa essere sfruttato per creare canali di comunicazione nascosti efficaci.

Durante lo sviluppo sono state implementate diverse tipologie di covert channel:

- Storage Covert Channel implementato sfruttando i vari campi dei messaggi ICMP.
- Timing Covert Channel codificando le informazioni negli intervalli temporali tra i pacchetti. Per poter inviare un maggior numero di

CONCLUSIONI SUI TEST EFFETTUATI

Nella trasmissione che invia una singola volta i dati, RITA non ha rilevato la presenza di un covert Channel. Ma questo caso non è applicabile alla realtà; in cui l'attaccante e la vittima si possono scambiare più volte i dati. Si è quindi proceduto a testare molteplici invii dei dati.

Le conclusioni sono che l'utilizzo di un delay fra l'invio di un blocco di dati ed il successivo permette di abbassare la visibilità del Covert Channel. Anche la falsificazione dei mittenti permette questa cosa, a patto che gli indirizzi IP utilizzabili non vengano utilizzati in sequenza per ogni pacchetto inviato ma a blocchi. Così da poter permettere agli IP precedentemente utilizzati di abbassare la loro visibilità.

In sintesi la rilevabilità di un Covert channel dipende da:

- Frequenza di trasmissione: Trasmissioni sporadiche sono più difficili da rilevare rispetto a comunicazioni con pattern regolari.

CONCLUSIONI SULL'UTILIZZO DI ICMP

Sebbene sia un protocollo funzionale al monitoraggio di rete, i suoi messaggi devono essere limitati a quelli essenziali. In particolare le principali tipologie di messaggi ICMP che servono sono la tipologia Echo Request/Reply (usata in strumenti come ping), la tipologia Destination Unreachable e la tipologia Time Exceeded. Il resto delle tipologie sono state deprecate siccome sostituite da strumenti diversi.

Inoltre fra quelli autorizzati bisogna monitorare il loro utilizzo ed il loro contenuto. Un messaggio Echo Request può contenere dati nel campo payload e messaggi come Destination Unreachable o Time Exceeded possono generare dei pattern temporali.

Ciò richiede un'ispezione dei pacchetti presenti nel flusso, un'analisi statistica per la ricerca di pattern e la limitazione (nella rete) di messaggi ICMP.